

CNN y GRU con Keras: diseño, evaluación y análisis de sesgo y varianza

Trabajo Práctico Integrador: Redes Neuronales | Inteligencia Artificial, Licenciatura en Sistemas de Información, FCyT-UADER

Integrantes: Facundo Tomás Hernández, Facundo Thomas Ramirez | Docentes: BiolIng. Alejandro Hadad, AS. Joaquín López y Mg. Santiago Díaz | Entrega: 30 de septiembre de 2026

1. Objetivo y protocolo

Se implementaron dos modelos independientes: una CNN que clasifica dígitos manuscritos y una GRU que pronostica un paso de una serie multivariada. Todas las decisiones (épocas, stride, tipo de celda) se tomaron con datos de desarrollo o validación; cada conjunto de prueba se evaluó una sola vez, con los pesos ya fijados. Se fijaron semillas y se exportaron los modelos completos y sus pesos.

2. Módulo CNN: datos y partición

- **Datos:** 1797 dígitos de scikit-learn (10 clases), interpolados de 8 x 8 a 32 x 32 y repartidos en 12 lotes de adquisición simulados, cada uno con brillo, contraste y desenfoque propios.
- **Partición:** lotes 0 a 7 para desarrollo (1207 imágenes) y 8 a 11 para prueba (590). Dentro de desarrollo, GroupKFold de 4 particiones: cada fold valida con dos lotes completos que el modelo no vio. Ninguna imagen ni lote cruza las fronteras.
- **Qué mide:** cada lote tiene las diez clases en proporciones similares, así que la validación por grupos mide robustez ante condiciones de captura no vistas. Como no hay identificadores de autor, no prueba generalización a escritores nuevos.

3. Arquitectura CNN y justificación

Tres convoluciones 3 x 3 con padding 'same', dos max pooling y global average pooling. El tamaño de cada mapa sale de:

$$O = \text{floor}((H + 2P - K) / S) + 1$$

Con $K = 3$ y $P = 1$, $S = 1$ conserva el tamaño y $S = 2$ lo divide por dos; el pooling ($K = 2$, $S = 2$, $P = 0$) también lo divide por dos. Así se obtiene la secuencia 32, 16, 8 y 4. Como la convolución comparte los mismos pesos en todas las posiciones, cada capa tiene:

$$\text{parámetros} = (K \cdot K \cdot C_{\text{entrada}} + 1) \cdot C_{\text{salida}} \quad \text{por ejemplo } (3 \cdot 3 \cdot 32 + 1) \cdot 64 = 18.496$$

Esa cantidad no depende del tamaño de la imagen: una capa densa entre esos mismos mapas (16 x 16 x 32 entradas y 16 x 16 x 64 salidas) necesitaría más de 134 millones de pesos. El campo receptivo, es decir cuántos píxeles de la imagen original ve cada unidad, crece capa a capa con:

$$r_i = r_{i-1} + (K - 1) \cdot j_{i-1} \quad \text{con } j_i = j_{i-1} \cdot S \text{ (distancia entre unidades vecinas)}$$

Capa	Salida	Parámetros	Campo receptivo
Conv2D 32 filtros 3x3, S=1, same	32 x 32 x 32	320	3 x 3
MaxPooling 2x2, S=2	16 x 16 x 32	0	4 x 4
Conv2D 64 filtros 3x3, S=1, same	16 x 16 x 64	18.496	8 x 8
MaxPooling 2x2, S=2	8 x 8 x 64	0	10 x 10
Conv2D 64 filtros 3x3, S=2, same	4 x 4 x 64	36.928	18 x 18
Global average pooling	64	0	-
Densa 64, ReLU	64	4.160	-
Dropout 0,25	64	0	-
Densa 10, softmax	10	650	-

Total: 60.554 parámetros entrenables. Cada unidad de la última convolución ve 18 x 18 píxeles. suficiente para abarcar el trazo del dígito.

- **Stride 2 en la última convolución:** reemplaza un tercer pooling por un submuestreo aprendido y evalúa 16 posiciones en lugar de 64. Rinde igual que $S = 1$ (0,9751 contra 0,9752 en validación) con menos dispersión entre lotes (0,0050 contra 0,0118).

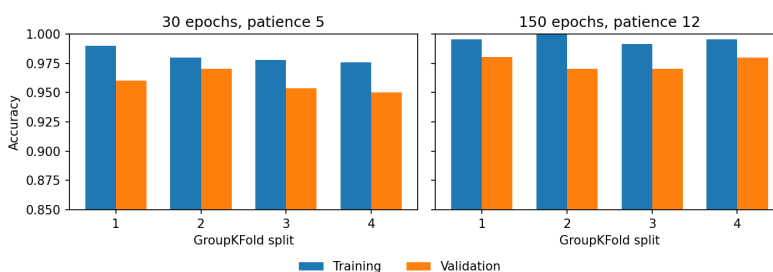
- **Pooling y equivarianza parcial:** la convolución con $S = 1$ es equivariante a traslaciones (desplazar la entrada desplaza la salida). El pooling y el stride 2 la conservan solo para desplazamientos múltiplos del paso y agregan invariancia local a corrimientos pequeños.
- **ReLU:** su derivada es 1 para entradas positivas, así que no satura ni atenúa el gradiente entre capas.
- **Softmax con entropía cruzada:** el gradiente respecto de cada logit es $p_k - y_k$, acotado entre -1 y 1.
- **Global average pooling:** la capa densa recibe 64 valores en lugar de 1.024, con muchos menos parámetros que aplanar.
- **Hiperparámetros:** dropout 0,25; Adam con tasa 0,001; lotes de 32; recorte de la norma del gradiente a 1,0; early stopping sobre la pérdida de validación con paciencia 12 y máximo de 150 épocas, restituyendo los mejores pesos.

4. CNN: análisis de sesgo y varianza

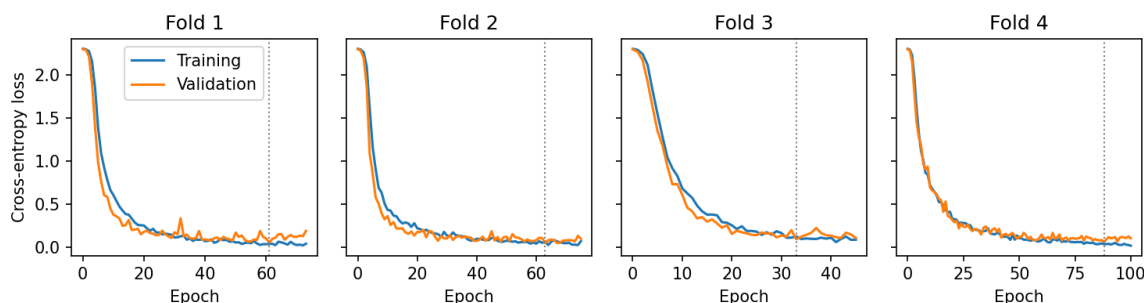
Aquí sesgo se usa en sentido estadístico (error sistemático por subajuste), no como sesgo inductivo de la arquitectura. Se compararon tres configuraciones con las mismas particiones y una referencia lineal (regresión logística sobre los píxeles). La brecha es entrenamiento menos validación.

Configuración	Mejores épocas	Entrenamiento	Validación	Brecha
Presupuesto corto: 30 épocas, paciencia 5	30, 28, 30, 30	0,9809	0,9586 $\pm$ 0,0077	0,0224
Elegida: 150 épocas, paciencia 12, $S = 2$	62, 64, 34, 89	0,9956	0,9751 $\pm$ 0,0050	0,0205
Variante: 150 épocas, paciencia 12, $S = 1$	48, 58, 51, 85	0,9868	0,9752 $\pm$ 0,0118	0,0116
Referencia: regresión logística	-	-	0,9602	-

- **Sesgo:** con el presupuesto corto los cuatro folds llegan al tope de 30 épocas (mejores épocas 28 a 30) y la validación (0,9586) queda por debajo de la regresión logística (0,9602). Es subajuste por entrenamiento incompleto, no por falta de capacidad. Con la configuración elegida la validación sube a 0,9751.
- **Varianza:** la brecha es de 2,1 puntos porcentuales. En las curvas, la pérdida de validación llega a un mínimo (línea punteada) y luego oscila o sube: es el inicio del sobreajuste, que early stopping corta. La dispersión entre folds (0,0050) es baja.
- **Prueba:** el modelo final se reentrenó con todo el desarrollo durante 63 épocas (mediana de las mejores). Sobre los lotes 8 a 11 obtuvo exactitud 0,9763 y F1 macro 0,9764 (14 errores en 590 imágenes), en línea con la validación cruzada. Las confusiones más frecuentes son 5 tomado como 9 y 3 tomado como 9.



Exactitud de entrenamiento y validación por fold, con presupuesto corto y con el elegido.



Pérdida por época en cada fold de la configuración elegida. La línea punteada marca la época restituida por early stopping.

5. Módulo GRU: serie y corte temporal

- **Serie:** 1600 instantes sintéticos con cuatro variables: el objetivo (autorregresivo), seno y coseno de período 48 y un control correlacionado, más ruido de desviación 0,11. El objetivo en t depende de su valor en $t-1$, de la estación y del control en $t-1$.
- **Entrada y salida:** los 36 pasos anteriores de las cuatro variables predicen el valor siguiente del objetivo.
- **Corte cronológico:** entrenamiento [0, 960), validación [960, 1280) y prueba [1280, 1600). Cada ventana queda completa dentro de su tramo y la normalización se ajusta solo con entrenamiento (gru_scalers.npz), así el futuro no contamina el pasado.

6. GRU: estabilidad del gradiente y elección de la celda

En BPTT, el gradiente hacia un estado pasado es un producto de jacobianos:

$$dL/dh_k = dL/dh_T \cdot J_T \cdot J_{T-1} \cdots J_{k+1} \quad \text{con } J_t = dh_t / dh_{t-1}$$

- **RNN simple:** $h_t = \tanh(W x_t + U h_{t-1})$, entonces $J_t = \text{diag}(1 - h_t^2) \cdot U$ y el producto queda acotado por $\|U\|^{T-k}$. Si $\|U\| < 1$ el gradiente desaparece; si es mayor, puede explotar.
- **GRU:** con una compuerta de actualización $z_t = \text{sigmoid}(W_z x_t + U_z h_{t-1})$ y un candidato g_t , el estado se combina como:

$$h_t = z_t \cdot h_{t-1} + (1 - z_t) \cdot g_t \quad \text{por lo tanto } J_t = \text{diag}(z_t) + (\text{términos por } g_t \text{ y } z_t)$$

El término $\text{diag}(z_t)$ es una vía aditiva que no se multiplica por U ni por la derivada de $\tanh$: si la red aprende z_t cercano a 1, el gradiente atraviesa muchos pasos sin atenuarse. Mitiga la desaparición, aunque no la elimina. Contra la explosión se recorta la norma del gradiente a 1,0.

Para evaluar la elección, las tres celdas se entrenaron con el mismo protocolo y se eligió por MAE de validación:

Celda	Parámetros	Mejor época	MAE validación	MAE prueba	RMSE prueba
GRU (elegida)	4.193	45	0,0891	0,0960	0,1201
LSTM	5.281	54	0,0949	0,1000	0,1244
RNN simple	1.729	24	0,0896	0,0981	0,1223

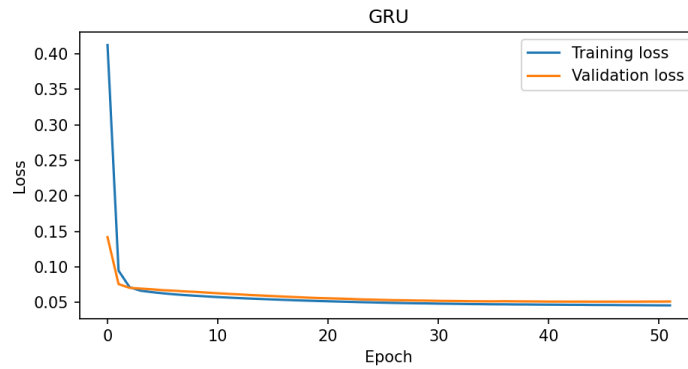
- **Resultado:** la GRU tiene el menor error de validación con 21% menos parámetros que la LSTM. La RNN simple queda cerca porque esta serie tiene dependencia corta: aquí el gradiente no necesita recorrer muchos pasos. La ventaja de las compuertas aparece con dependencias largas, y la GRU la ofrece con menos parámetros.
- **Hiperparámetros:** 32 unidades (tanh en el estado, sigmoid en las compuertas), densa de 16 unidades ReLU y salida lineal, porque es una regresión sin rango fijo. Pérdida MSE, Adam con tasa 0,001, lotes de 32, orden cronológico conservado y early stopping con paciencia 7 (máximo 55 épocas). La ventana de 36 pasos cubre tres cuartos del ciclo estacional.

7. GRU: resultados y sesgo y varianza

Como la serie es sintética se conoce su parte predecible: la ecuación verdadera solo falla por el ruido, lo que fija un piso de error. La persistencia (repetir el último valor) es la referencia ingenua.

Modelo	MAE prueba	RMSE prueba
GRU	0,0960	0,1201
Persistencia	0,1139	0,1428
Piso de ruido (ecuación verdadera)	0,0917	0,1150

- **Sesgo:** la GRU mejora 15,7% a la persistencia y queda solo 4,7% por encima del piso: casi todo el error restante es ruido irreducible.
- **Varianza:** MAE de 0,0841 en entrenamiento, 0,0891 en validación y 0,0960 en prueba. Las brechas son pequeñas y las curvas convergen sin divergir.
- **Estabilidad predictiva:** el MAE por cuartos cronológicos de la prueba es 0,0987, 0,1002, 0,0947, 0,0902; no crece al alejarse del entrenamiento.



Pérdida MSE por época de la GRU en entrenamiento y validación (mejor época: 45).

8. Limitaciones y reproducción

Los lotes son simulados y la serie es sintética y de dependencia corta: sirven para verificar el protocolo, pero no se extrapolan a datos reales. Para reproducir: `pip install -r requirements.txt`, `python train.py`, `python predict.py`, `python -m unittest discover -s tests` y `python make_report.py`. Los modelos están en `artifacts/` (`cnn.keras`, `gru.keras` y sus `.weights.h5`) junto con todas las métricas.

Referencias: documentación de Keras (keras.io/api) y scikit-learn (scikit-learn.org/stable); Goodfellow, Bengio y Courville, Deep Learning, MIT Press, 2016, capítulos 9 y 10.